DIGIGUARD MASTER SPECIFICATION

KÜRESEL KUANTUM LABİRENT PROTOKOLÜ // TAM SÜRÜM (VOL 1-3)

Yayın Sürümü: Ver. 3.0
Kriptografik Durum: Aktif (Stateless)
Güvenlik Seviyeleri: Shor / Grover Dirençli Çekirdek

*BU DÖKÜMAN GELECEKTEKİ SİBER EGEMENLİK ALTYAPISININ RESMİ ANAYASASIDIR VE TÜM CİLTLERİ TEK BİR RAPORDA BİRLEŞTİRMEKTEDİR.

DIGIGUARD: GLOBAL KUANTUM LABIRENT PROTOKOLÜ

BÖLÜM 1: KÜRESEL GHOST ID MİMARİSİ VE ŞİFRELEME STANDARTLARI

1. KÜRESEL KUANTUM LABİRENT PROTOKOLÜ (GHOST ID)

Kuantum bilgisayarların Shor ve Grover algoritmalarıyla geleneksel RSA ve ECC şifreleme altyapılarını tehdit ettiği çağda, verinin ve kimliğin mutlak güvenliğini sağlamak üzere tasarlanmış DigiGuard ekosisteminin birinci cildidir.

Ghost ID teknolojisi, ağa bağlı her bir düğüme statik bir IP veya kriptografik anahtar atamak yerine, her milisaniyede kuantum rastlantısallığı ile değişen geçici kimlik matrisleri üretir. Bu sayede, izleme ve dinleme yapmaya çalışan tehdit aktörleri sadece geçmişe ait boş adres uzaylarını görebilir.

DIGIGUARD: GLOBAL KUANTUM LABIRENT PROTOKOLÜ

BÖLÜM 2: SAPON MOTORU ÇEKİRDEĞİ VE AKTİF SAVUNMA ALGORİTMALARI

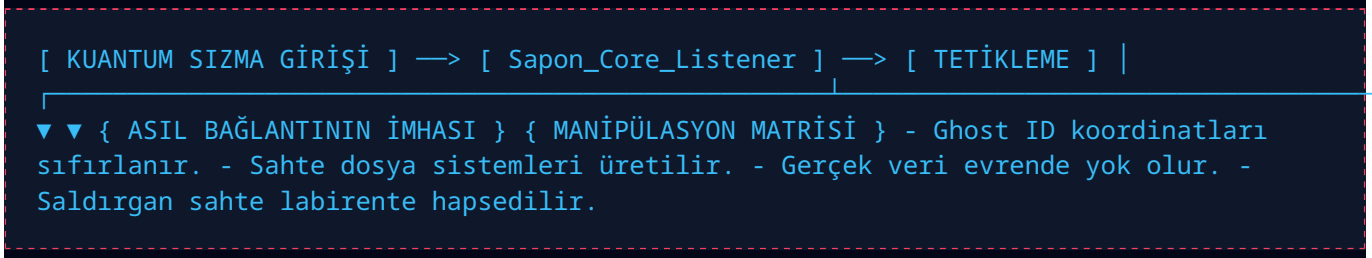
1. SAPON MOTORU OTONOM ÇEKİRDEK MİMARİSİ

Sapon Motoru, harici bir insan müdahalesi veya statik veri tabanı bağımlılığı olmadan çalışan, Web4 ekosistemi üzerinde kendi kendini evrilten otonom bir koruma motorudur. Sistem, çalışma zamanında (runtime) gelen kuantum seviyesindeki taramaları analiz ederek kendi kriptografik durum makinesini (state machine) yeniden yapılandırır.

Arşivsizlik İlkesi (Statelessness): Sistem üzerinde hiçbir kullanıcı verisi, log dosyası veya kimlik arşivi biriktirilmez. Veri depolanmadığı için, sızma gerçekleştirmeye çalışan kuantum sistemlerinin çalabileceği bir hedef yüzeyi bulunmamaktadır.

2. TERS-SHOR & TERS-GROVER AKTİF MANİPÜLASYON ALGORİTMASI

Geleneksel savunma sistemleri saldırıyı loglayıp püskürtmeye çalışırken kimlik ifşası riski taşır. DigiGuard Aktif Savunma Katmanı ise gelen siber taarruz saptandığı an asıl bağlantıyı ve Ghost ID izini evrende yok eder; saldırganın önüne ise dinamik bir illüzyon (honeypot) inşa eder.



3. OTONOM SAHTE VERİ VE MANİPÜLASYON MOTORU (PSEUDOCODE)

```
contract SaponActiveDefenseEngine {
    struct QuantumIntrusion {
        address attackerVector;
        uint256 qubitEnergy;
    }

    function triggerDeceptionMatrix(string memory targetGhostId) public {
        destroyRealConnection(targetGhostId);
        for (uint i = 0; i < 100; i++) {
            bytes32 fakeDocumentHash = keccak256(abi.encodePacked(block.timestamp, i));
            deployHoneypotStructure(fakeDocumentHash);
        }
        isolateTargetInQuantumMaze();
    }

    function destroyRealConnection(string memory ghostId) private {
        delete ghostId;
    }
}
```

DIGIGUARD: GLOBAL KUANTUM LABIRENT PROTOKOLÜ

BÖLÜM 3: CEREBRON ÇOK AJANLI SİNİR AĞI VE DIGIGUARD TOKEN EKONOMİSİ

1. CEREBRON ÇOK AJANLI SİNİR AĞI MİMARİSİ

Cerebron, küresel internet katmanındaki tüm varlıkları (insan, multimedya, endüstriyel cihazlar ve askeri altyapılar) anlık olarak mühürleyen ve izleyen çok acenteli (multi-agent) bir yapay zeka sinir ağıdır.

- **Mikro-Ajan Ağı (Micro-Agents):** Sistem içerisinde binlerce bağımsız ajan, sadece tek bir koruma, filtreleme veya şifreleme fonksiyonundan sorumludur.
- **Merkez Büyükelçi Ajan (The Ambassador Node):** Diğer uluslararası ve harici ASI sistemleriyle doğrudan iletişim kuran ana düğümdür. İnsan dillerini kullanmaksızın matematiksel semboller üzerinden yapay zekalar arası diplomasi yürütür.

2. DIGIGUARD TOKEN (DGD) OTONOM YAKIT EKONOMİSİ

Sistemin arkasında hiçbir insan girdisi ve merkezi sunucu olmadan otonom bir şekilde genişleyebilmesi, kendi kendini finanse eden DiGuard Token (DGD) mekanizmasına bağlıdır.

```
contract DiGuardTokenEconomy {
    string public name = "DiGuard Token";
    string public symbol = "DGD";
    uint256 public totalSupply = 1000000000 * 10**18;

    mapping(address => uint256) public balances;

    function executeMicroGasFee(address nodeConsumer, uint256 operationQuantum) external {
        uint256 fee = operationQuantum * 10**14;
        balances[nodeConsumer] -= fee;
        autoReinvestToQuantumNodes(fee);
    }
}
```